

Curso de Jogos Digitais

Disciplina de Computação Gráfica

Carregando Objetos no OpenGL Moderno

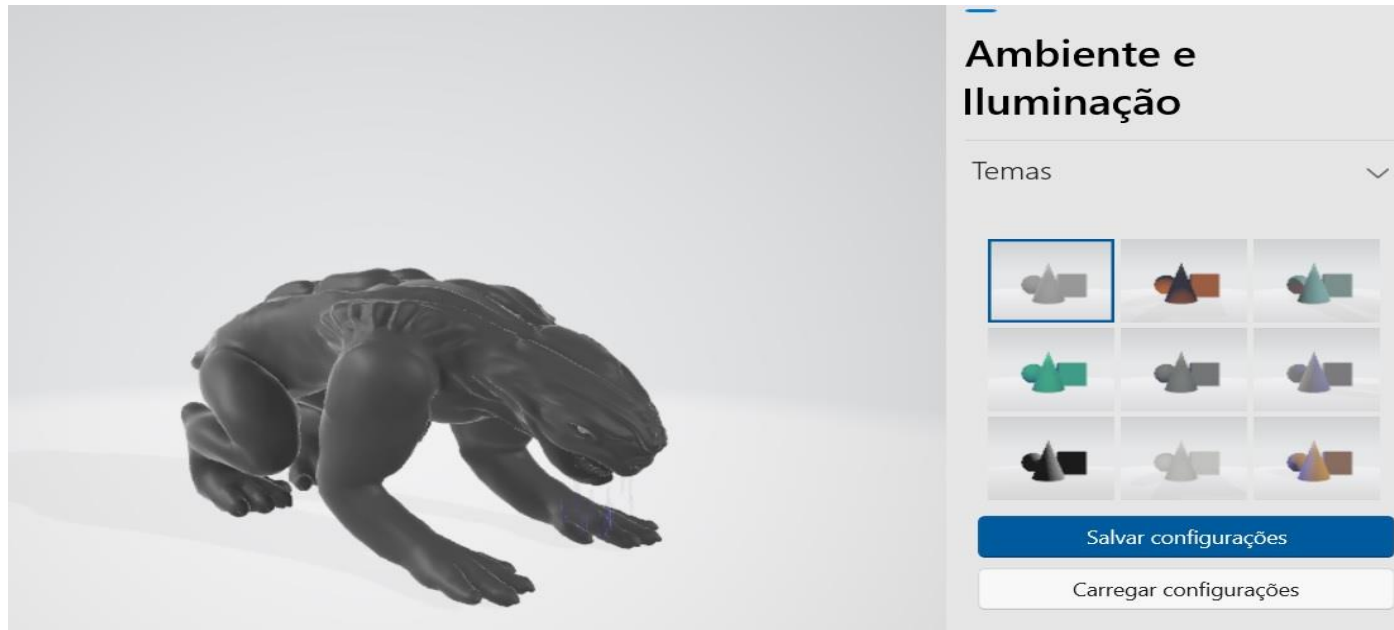
Aula 16

Professor: André Flores dos Santos



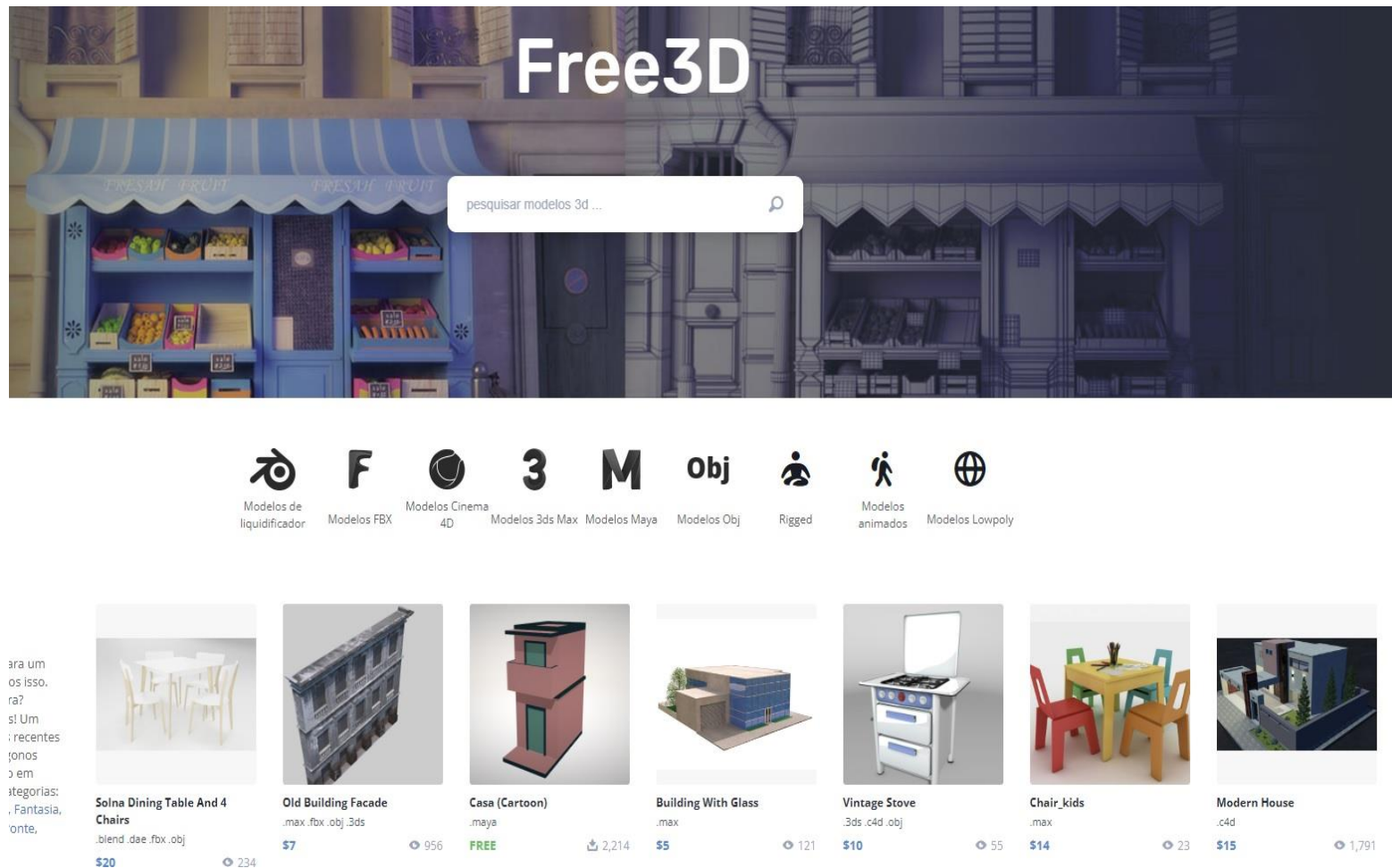
Introdução

- Objetivo principal:
 - Carregar objetos 3D prontos nos nossos trabalhos já desenvolvidos. Fazendo uma mescla de objetos criados e objetos prontos baixados de sites desse ramo.



Sites para baixar Objetos

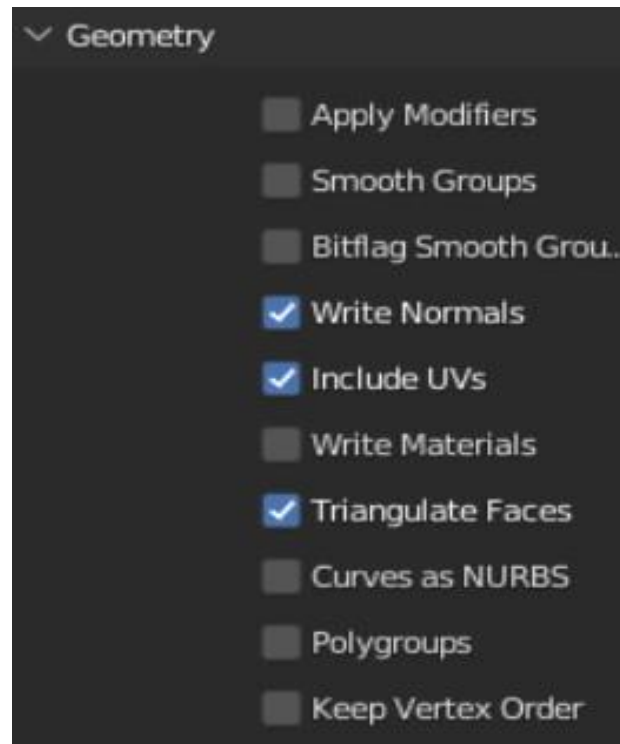
Bom site para baixar modelos 3d: <https://free3d.com/>



Sites para baixar Objetos

Dica:

Caso um arquivo Wavefront (.obj) encontrado na internet não funcione no exemplo abaixo, importe-o no **Blender**, e exporte marcando somente as opções abaixo:

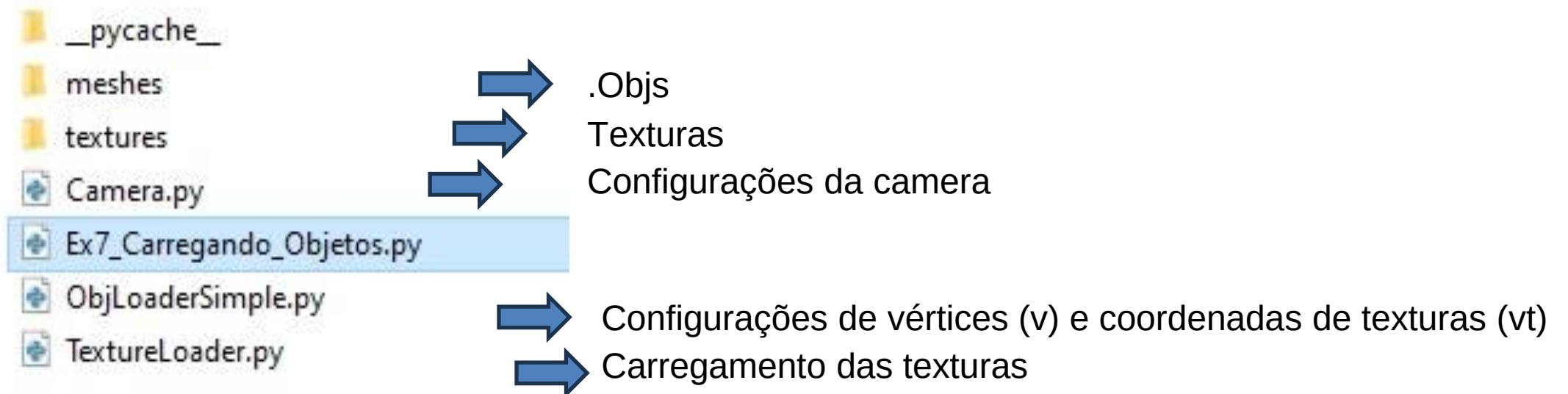


Código Base para o Trabalho 02

Estrutura de pastas para o exemplo.

Carregamos arquivos '*.obj' a partir da pasta 'meshes' e podemos aplicar texturas que podem ser usadas a partir da pasta 'textures'. Outros arquivos de configurações são 'Camera.py', 'ObjLoader.py' e 'TextureLoader.py'.

https://github.com/andreflores2009/ComputacaoGrafica_2025-01_JD/tree/main/Exercicios/Aula16



Instalação de bibliotecas

- Instalar as bibliotecas glfw, PyOpenGL, numpy, PYRRR e PILLOW:

Comando:

```
pip install glfw PyOpenGL numpy pyrr  
Pillow
```

Dicas para Carregar Objetos no nosso Código

Ex7_Carregando_Objeto.py

Configura a função do Objeto

```
108 # Carregamento de objeto e textura
109 # -----
110
111 def inicializa_objeto():
112     """
113     Carrega vértices e UVs via load_obj(),
114     cria VAO/VBO e configura atributos de vértice.
115     Também carrega a textura chibi.png.
116     """
117     global vao_objeto, num_vertices, obj_textura
118
119     # load_obj retorna (vertex_buffer, num_vertices)
120     buffer, num_vertices = ObjLoaderSimple.load_obj("meshes/chibi.obj")
121     buffer = buffer.astype(np.float32)
122
123     # Gera e vincula VAO
124     vao_objeto = glGenVertexArrays(1)
125     glBindVertexArray(vao_objeto)
126
127     # Gera VBO e envia dados
128     vbo = glGenBuffers(1)
129     glBindBuffer(GL_ARRAY_BUFFER, vbo)
130     glBufferData(GL_ARRAY_BUFFER, buffer.nbytes, buffer, GL_STATIC_DRAW)
131
132     # Cada vértice: 5 floats → [x,y,z, u,v]
133     stride = buffer.itemsize * 5
134     # Posição (vec3) no location=0
135     glEnableVertexAttribArray(0)
136     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, stride, ctypes.c_void_p(0))
137     # UV (vec2) no location=1
138     glEnableVertexAttribArray(1)
139     glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, stride, ctypes.c_void_p(buffer.itemsize * 3))
140
141     # Carrega textura
142     obj_textura = glGenTextures(1)
143     #load_texture("textures/chibi.png", obj_textura)
144     load_texture("textures/chibi.png", obj_textura)
145
146
147 # -----
148 # Carregamento de objeto e textura
```

Dicas para Carregar Objetos no nosso Código

Ex7_Carregando_Objeto.py

Função render_loop()

Envia as matrizes e desenha
o Objeto

```
# Envia uniforms
#localização do uniform
# 1, // quantidade de matrizes a enviar
# GL_FALSE // flag de "transpose" ou transposta
# model // ponteiro/data da matriz
glUniformMatrix4fv(glGetUniformLocation(Shader_programm, "model"), 1, GL_FALSE, model)
glUniformMatrix4fv(glGetUniformLocation(Shader_programm, "view"), 1, GL_FALSE, view)
glUniformMatrix4fv(glGetUniformLocation(Shader_programm, "projection"), 1, GL_FALSE, projection)

# Desenha o objeto
glBindVertexArray(vao_objeto)
glBindTexture(GL_TEXTURE_2D, obj_textura)
glDrawArrays(GL_TRIANGLES, 0, num_vertices)
```


Dicas para Carregar Objetos no nosso Código

Ex7_Carregando_Objeto.py

Configura a função
inicializa_gato() do Objeto
Gato

```
148 # Carregamento de objeto e textura
149 # -----
150
151 def inicializa_gato():
152     """
153     Carrega vértices e UVs via load_obj(),
154     cria VAO/VBO e configura atributos de vértice.
155     Também carrega a textura chibi.png.
156     """
157     global vao_gato, num_vertices_gato, obj_textura_gato
158
159     # load obj retorna (vertex buffer, num vertices)
160     buffer, num_vertices_gato = ObjLoaderSimple.load_obj("meshes/Cat/Cat.obj")
161     buffer = buffer.astype(np.float32)
162
163     # Gera e vincula VAO
164     vao_gato = glGenVertexArrays(1)
165     glBindVertexArray(vao_gato)
166
167     # Gera VBO e envia dados
168     vbo = glGenBuffers(1)
169     glBindBuffer(GL_ARRAY_BUFFER, vbo)
170     glBufferData(GL_ARRAY_BUFFER, buffer.nbytes, buffer, GL_STATIC_DRAW)
171
172     # Cada vértice: 5 floats → [x,y,z, u,v]
173     stride = buffer.itemsize * 5
174     # Posição (vec3) no location=0
175     glEnableVertexAttribArray(0)
176     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, stride, ctypes.c_void_p(0))
177     # UV (vec2) no location=1
178     glEnableVertexAttribArray(1)
179     glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, stride, ctypes.c_void_p(buffer.itemsize * 3))
180
181     # Carrega textura
182     obj_textura_gato = glGenTextures(1)
183     load_texture("textures/Cat_diffuse.jpg", obj_textura_gato)
184
185
```

Dicas para Carregar Objetos no nosso Código

Ex7_Carregando_Objetos.py

Função render_loop()

**Envia as matrizes e desenho
o Objeto Gato.**

**Precisamos fazer algumas
transformações para ajustar
o objeto gato na tela.**

```
# -----  
# Envia uniforms para o GATO  
  
# Cria matriz de translação: desloca para a direita  
trans_gato = pyrr.matrix44.create_from_translation(pyrr.Vector3([30.0, 0.0, 0.0]))  
  
# Cria matriz de rotação: gira 90 graus no eixo X  
rot_gato = pyrr.matrix44.create_from_x_rotation(np.radians(90))  
  
# Cria matriz de escala: reduz tamanho  
escala_gato = pyrr.matrix44.create_from_scale(pyrr.Vector3([0.2, 0.2, 0.2]))  
  
# Combina as transformações: Translação * Rotação * Escala  
model_gato = pyrr.matrix44.multiply(rot_gato, escala_gato)  
model_gato = pyrr.matrix44.multiply(trans_gato, model_gato)  
  
# Envia uniforms  
#localização do uniform  
# 1, // quantidade de matrizes a enviar  
# GL_FALSE // flag de "transpose" ou transposta  
# model_gato // ponteiro/data da matriz  
glUniformMatrix4fv(glGetUniformLocation(Shader_programm, "model"), 1, GL_FALSE, model_gato)  
glUniformMatrix4fv(glGetUniformLocation(Shader_programm, "view"), 1, GL_FALSE, view)  
glUniformMatrix4fv(glGetUniformLocation(Shader_programm, "projection"), 1, GL_FALSE, projection)  
  
# Desenha o gato  
glBindVertexArray(vao_gato)  
glBindTexture(GL_TEXTURE_2D, obj_textura_gato)  
glDrawArrays(GL_TRIANGLES, 0, num_vertices_gato)
```

TRABALHO 03. Envie na atividade da aula de hoje.

Peso (2,0)

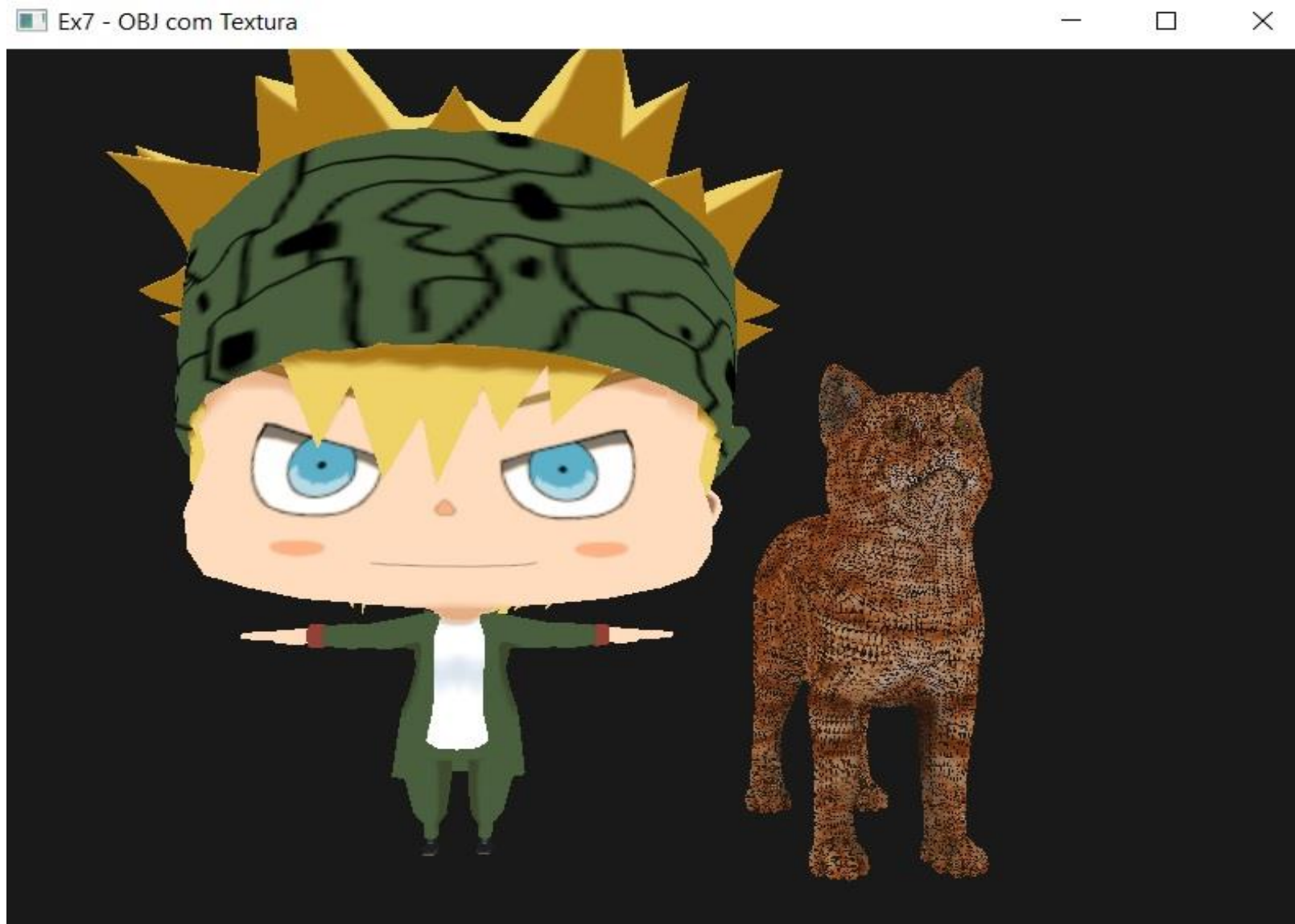
Carregar mais 5 Objetos.

Passos:

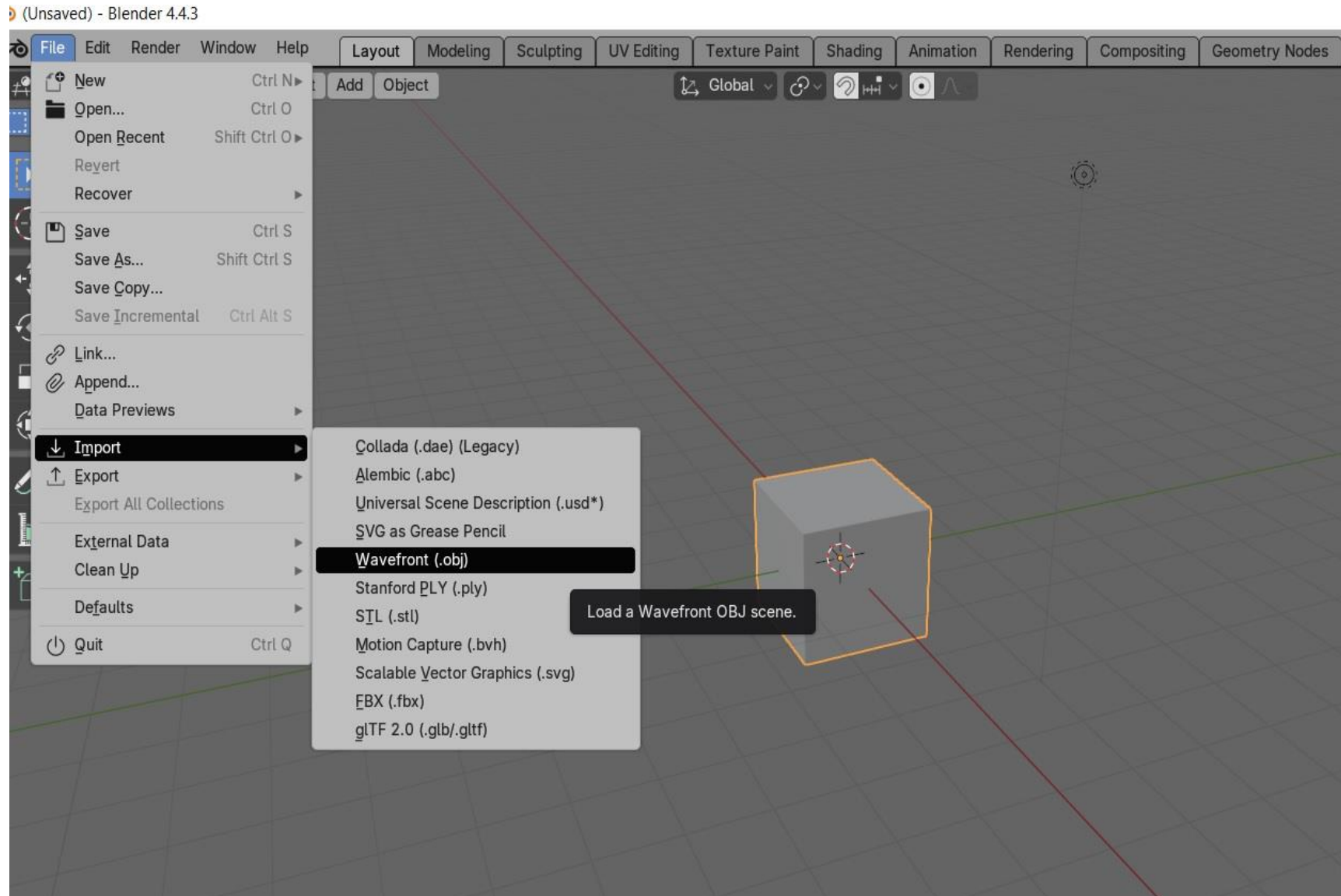
- 1- Criar uma função para cada objeto onde é configurado o caminho para carregar o Objeto e a Textura. É necessário configurar o nome das variáveis globais.
- 2- Na função **render_loop()**, logo após os comandos para carregar o gato, insira novos comandos para renderizar os próximos objetos, um por um.
- 3- Não esqueça de chamar na **main()** o nome das funções que você criou. Ex:

```
def main():  
    inicializa_opengl()  
    inicializa_objeto()  
    inicializa_gato()  
    inicializa_shaders()  
    render_loop()
```

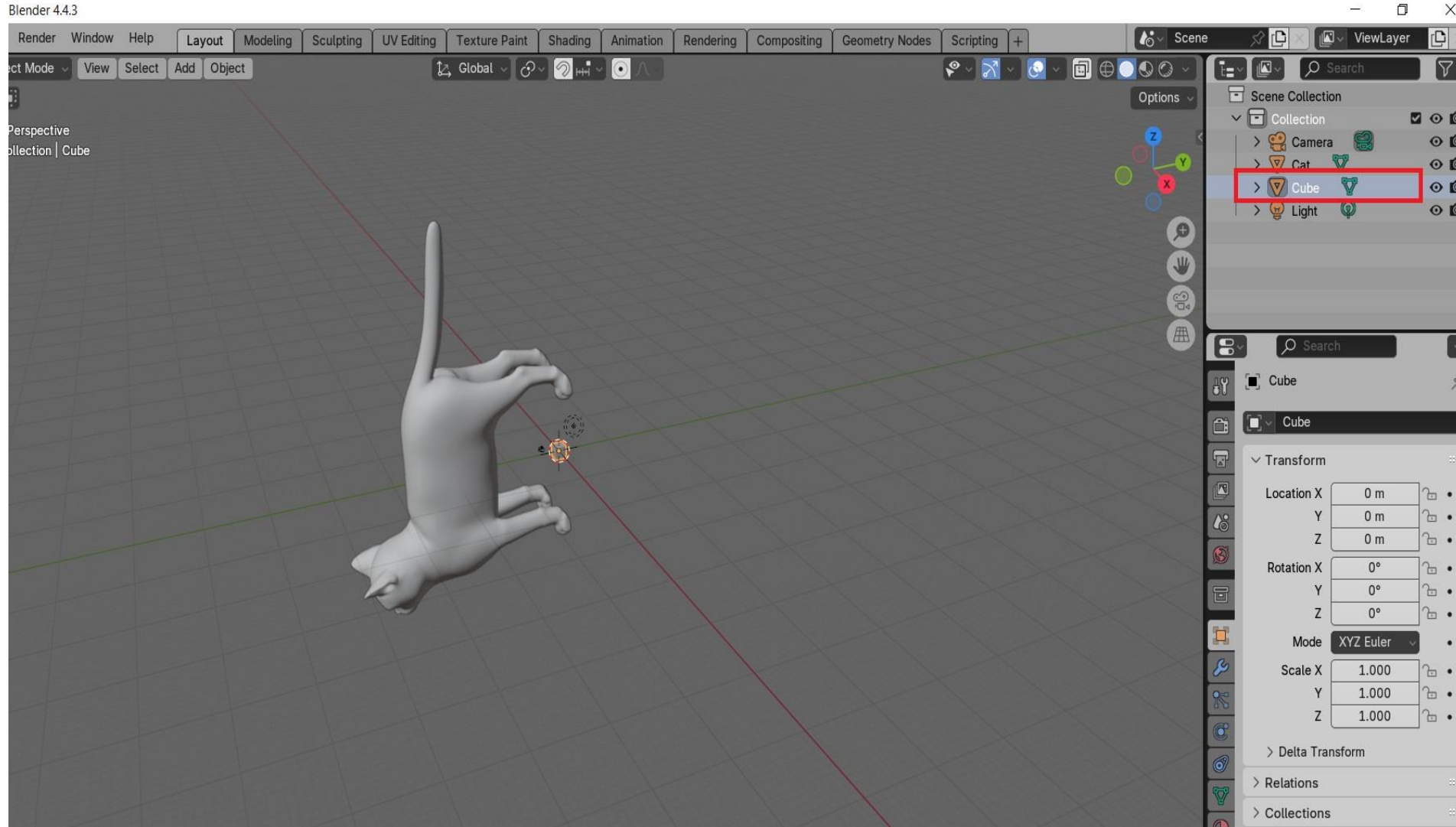
Resolvendo Problemas de Texturas com ‘Buracos’ no Blender.



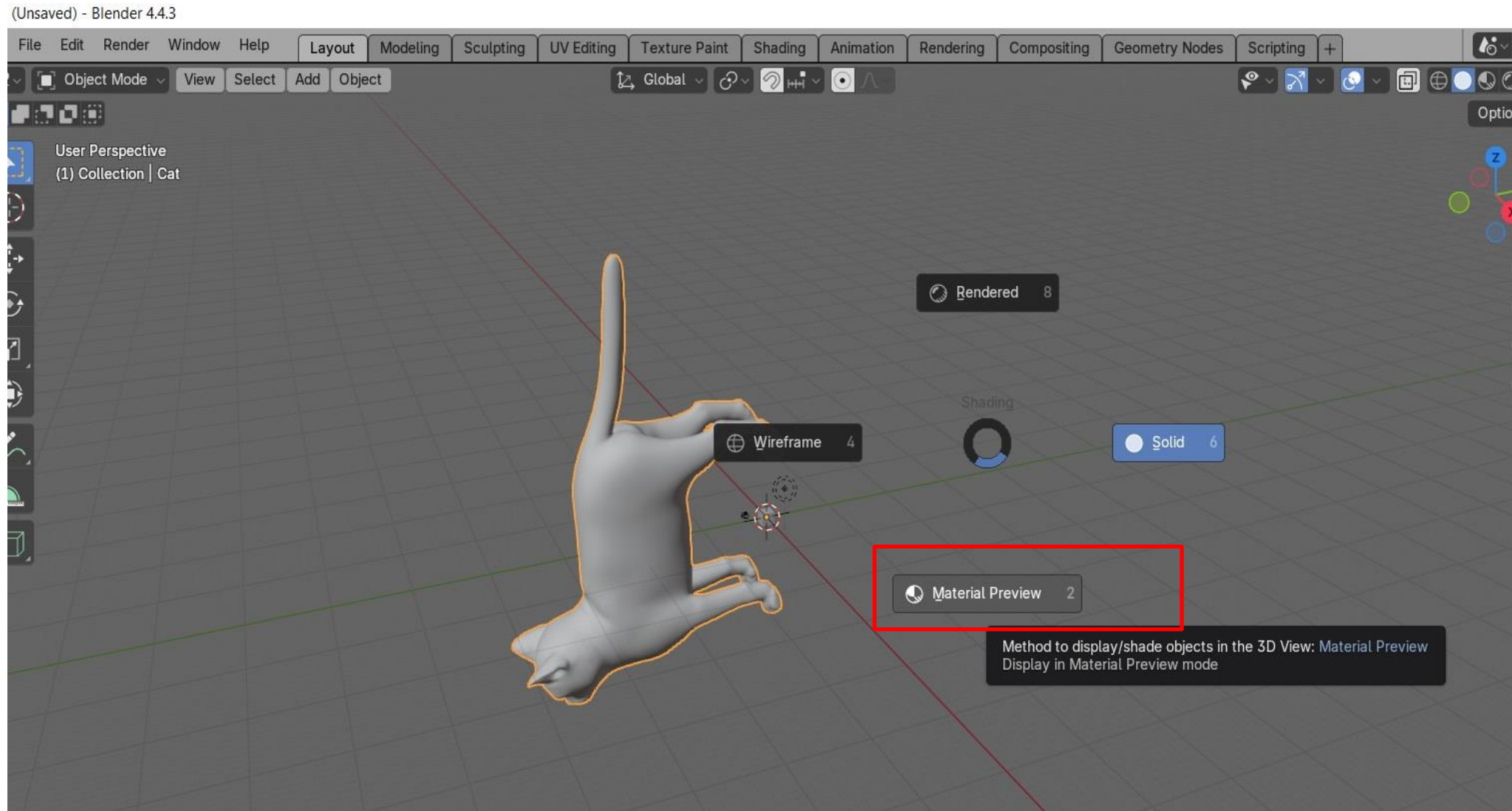
Abrir o 'arquivo.obj' no Blender, no caso 'Cat.obj'



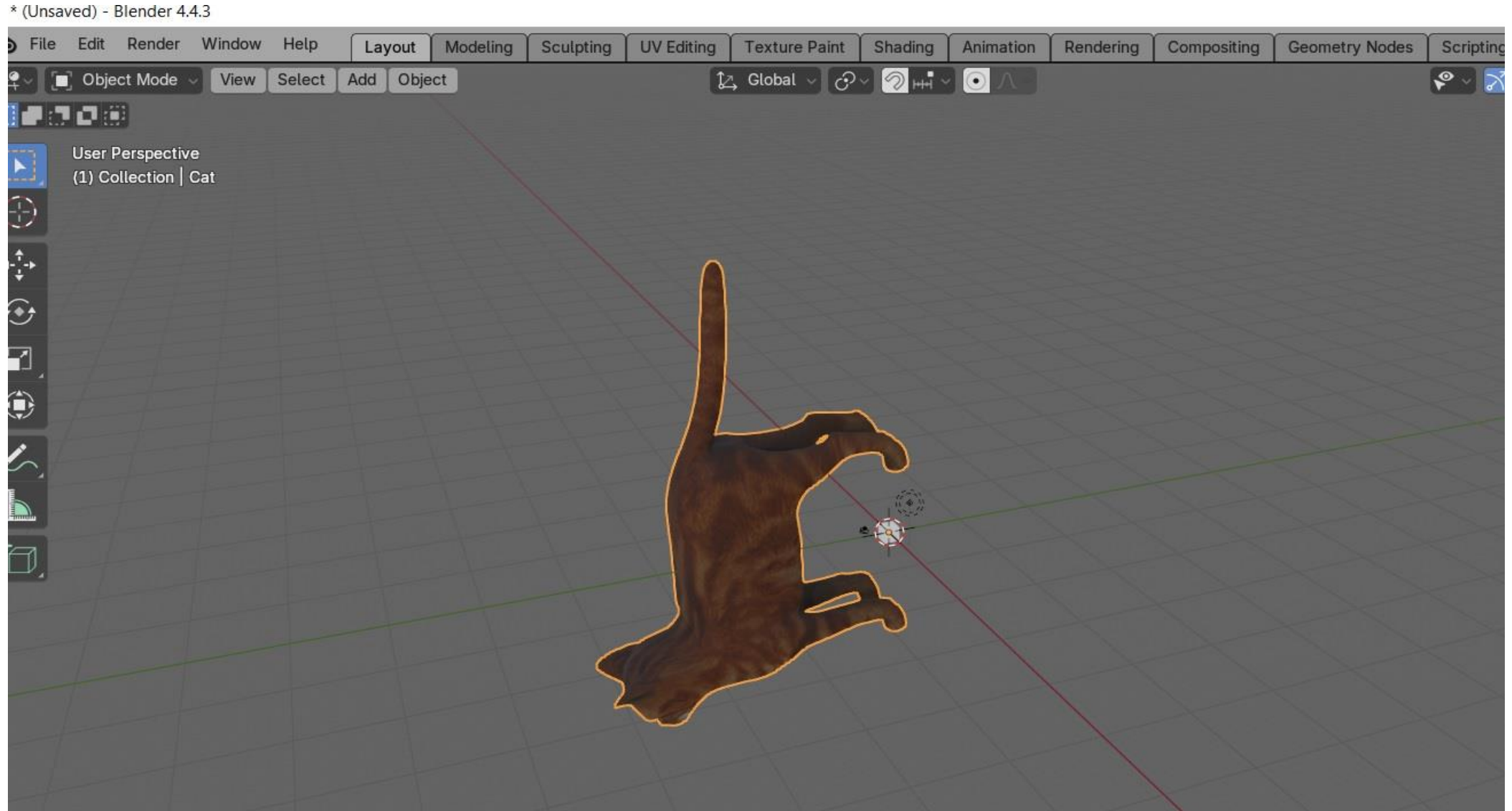
Delete o cubo padrão na lista de coleção da Cena. Se não ele será exportado junto depois.



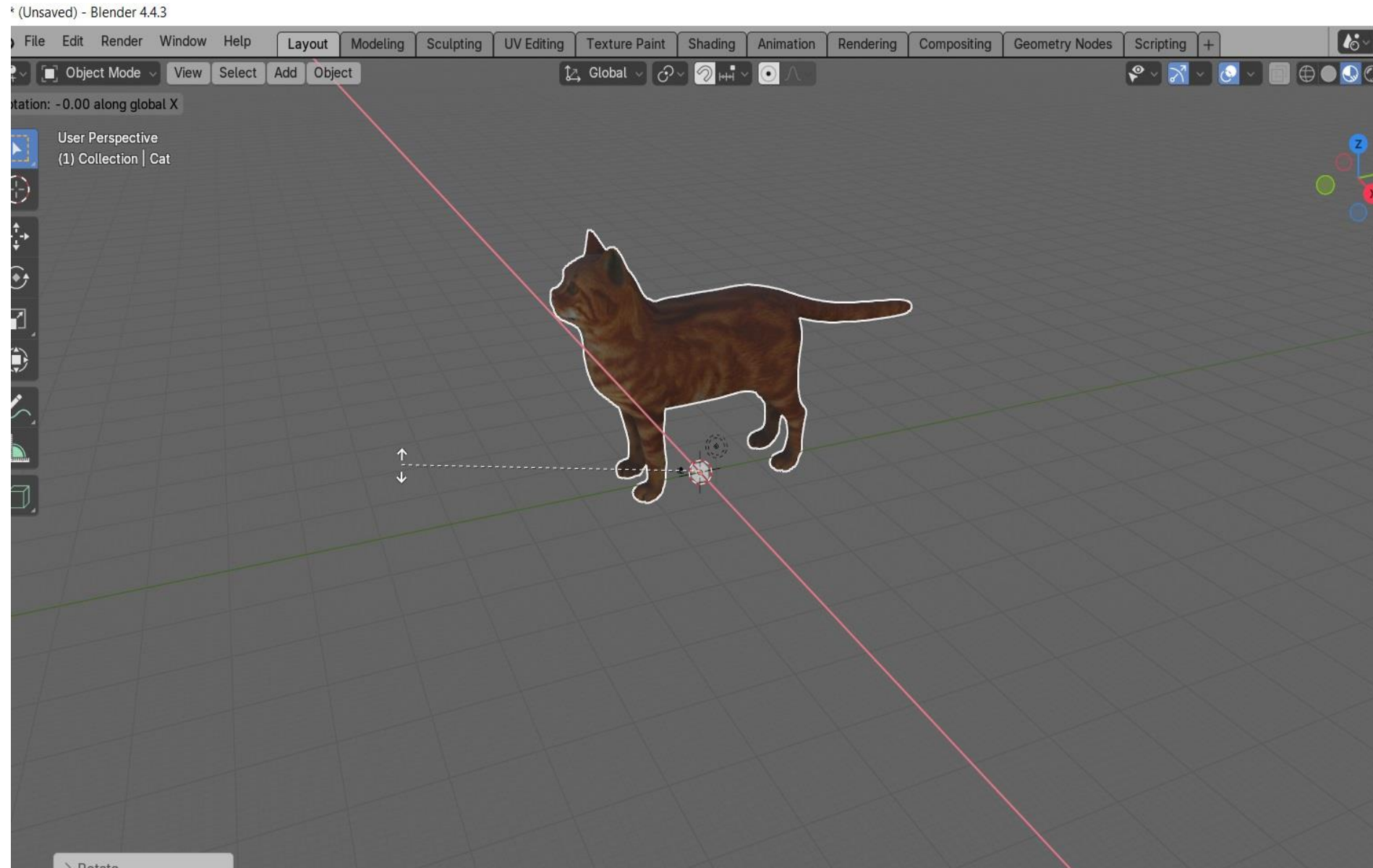
Aperte a tecla 'Z' e clique em 'Material Preview' para aparecer a textura do gato, se ela estiver corretamente vinculada na pasta.



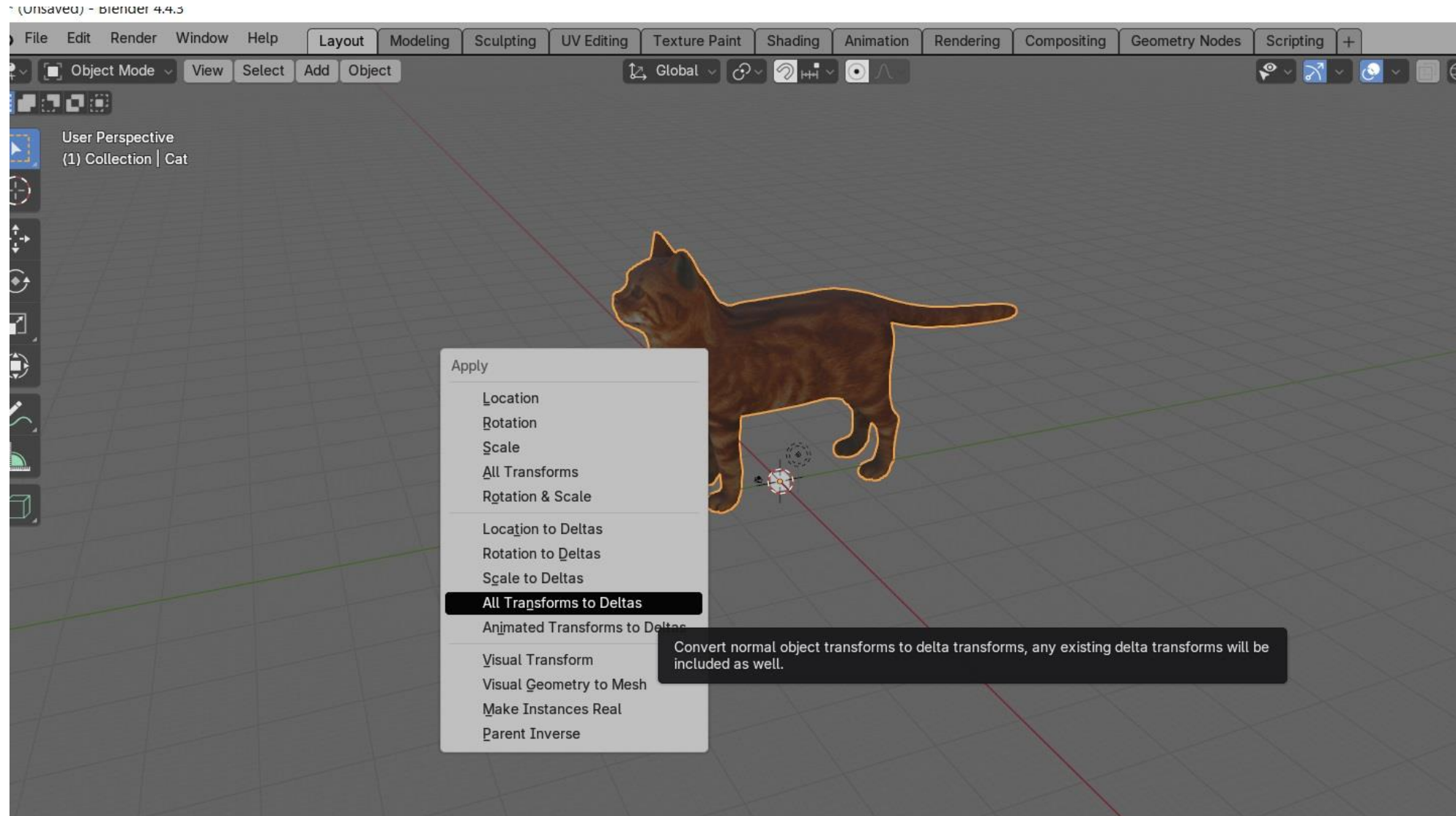
Se tudo estiver correto a textura irá aparecer. Agora podemos girar o gato e deixar na posição correta, ao invés de aplicar ‘rotação’ com comando no OpenGL. Quando salvar o Gato a posição se manterá.



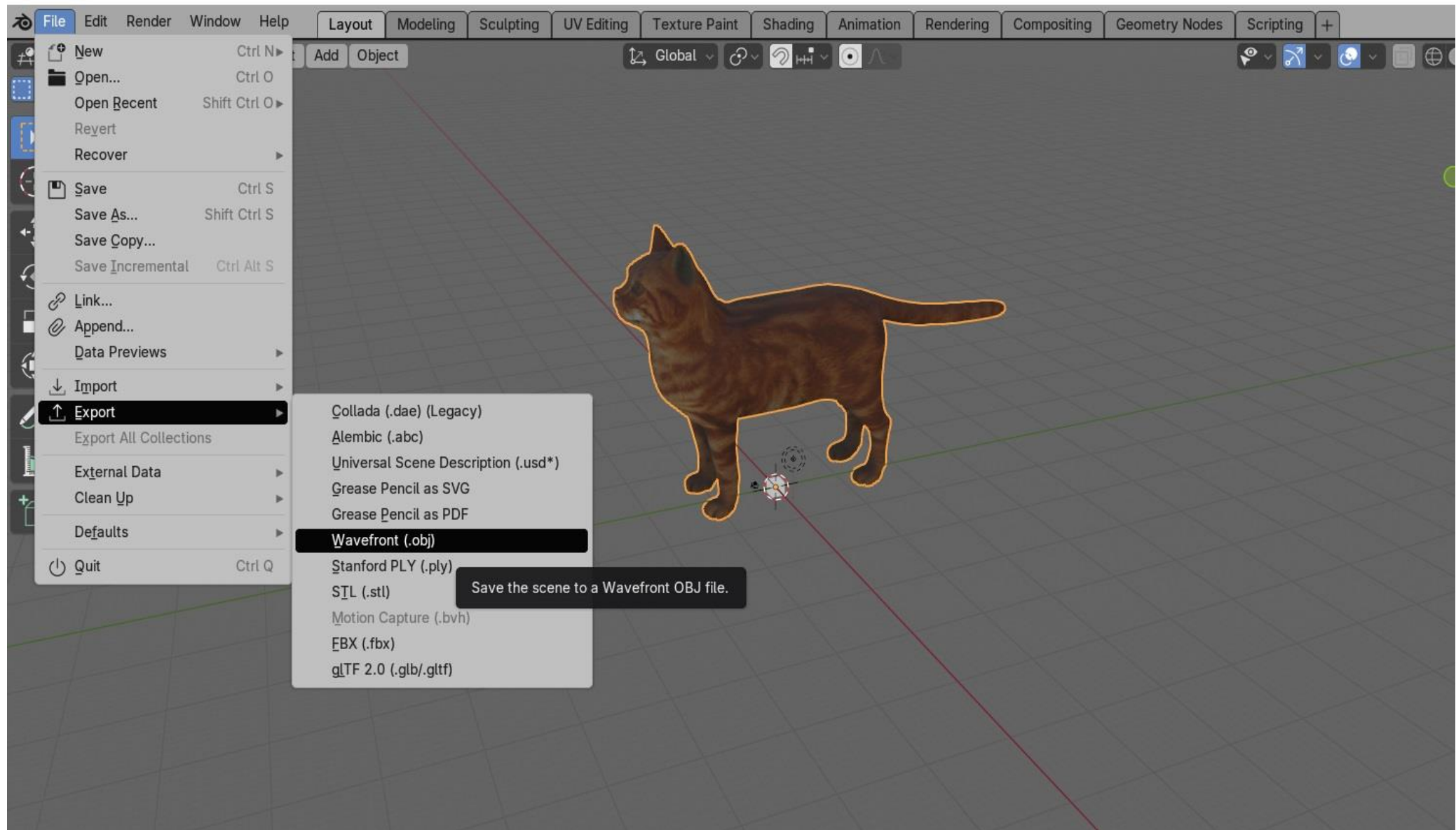
Com o Gato selecionado, pressione a tecla ‘r’ e depois ‘x’ e gire o gato com o mouse para a posição adequada.



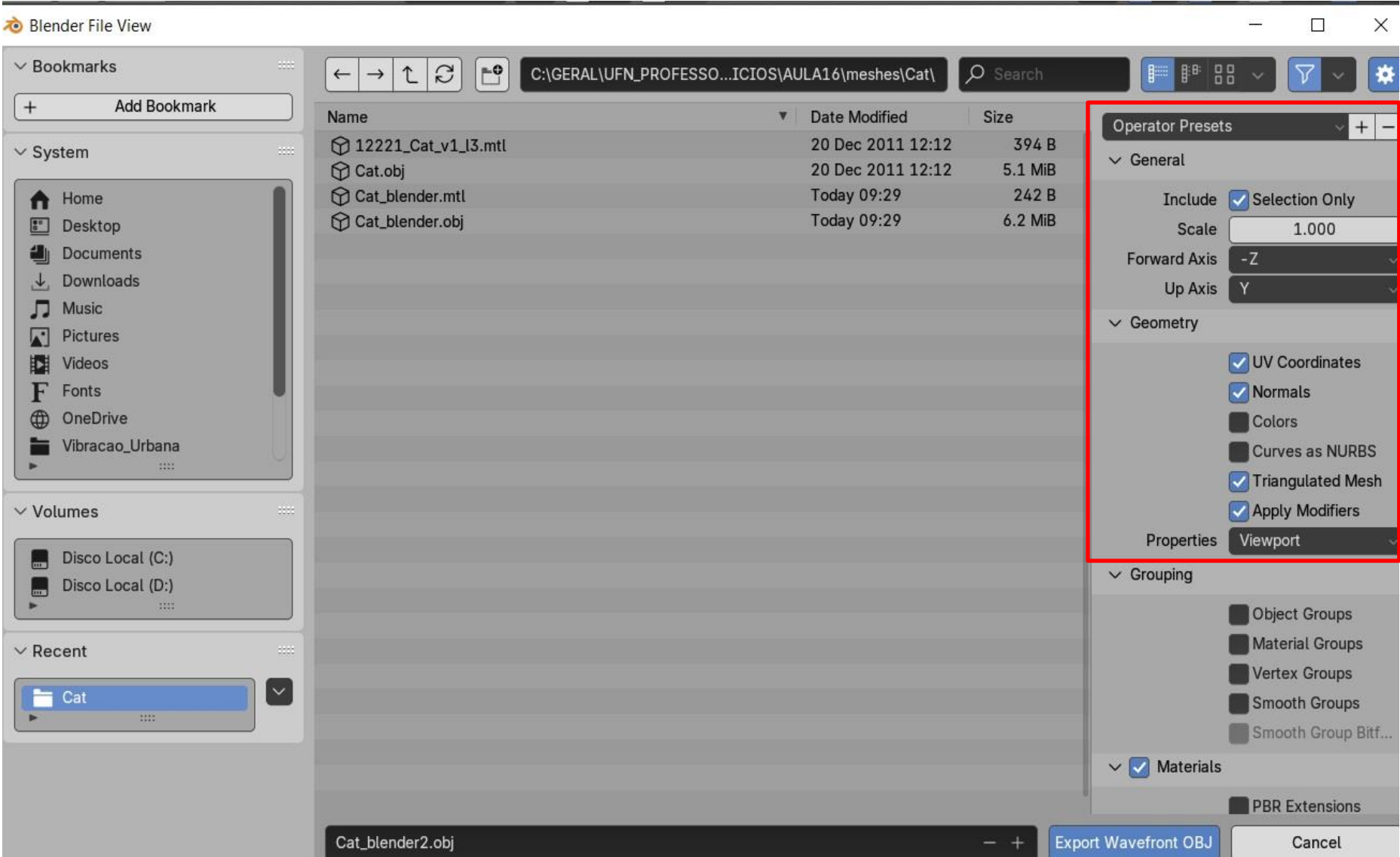
Para confirmar as transformações, aperte a tecla ‘Ctrl+a’ e clique em ‘All Transforms to Deltas’.



Vamos exportar o ‘Cat_blender.obj’.



Escolha o caminho para salvar e o nome.obj. Marque as opções de geometria corretas.

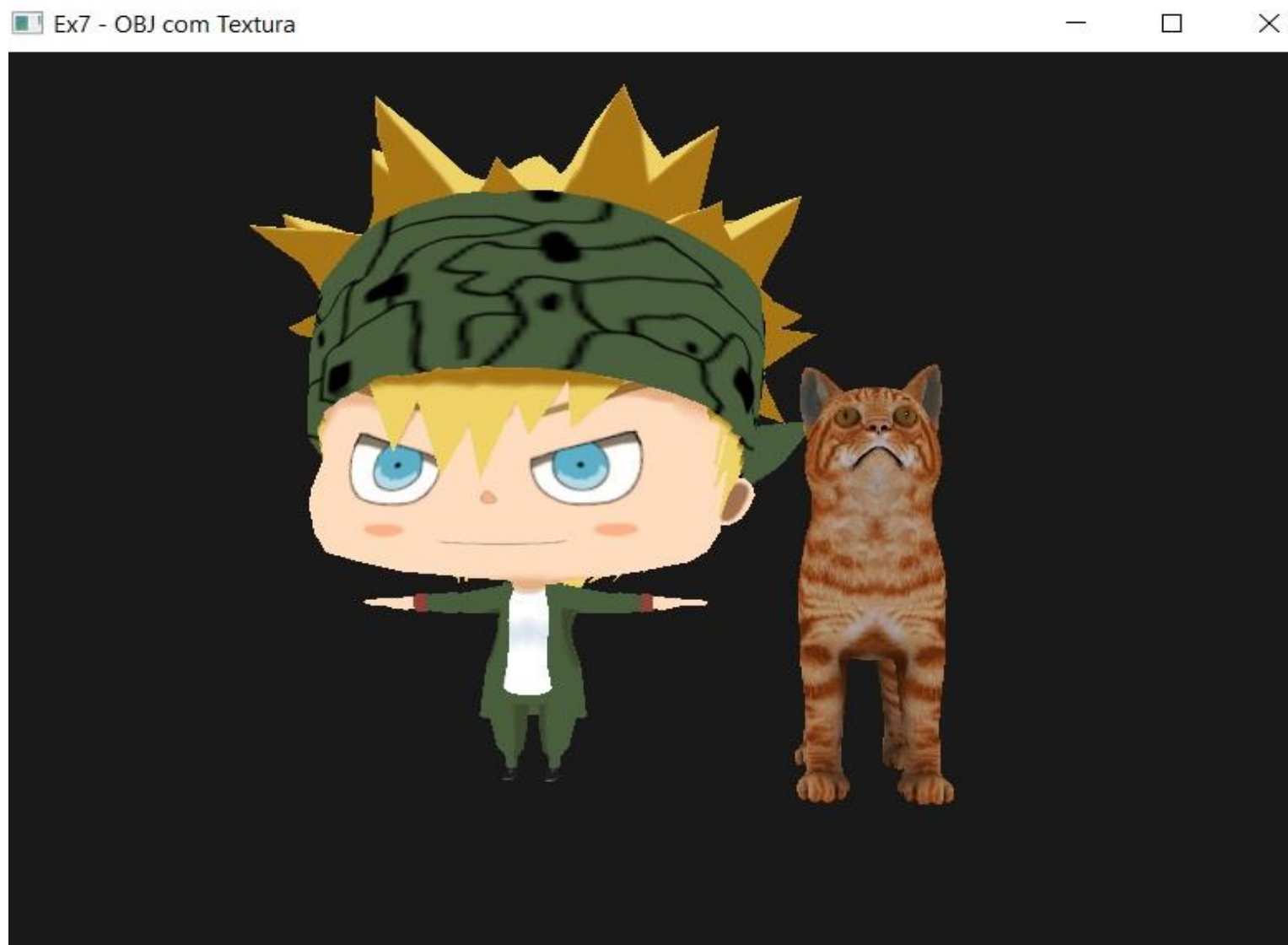


Atenção verifique se dentro do arquivo **‘Cat_blender.obj’** foi escrito os dados corretamente. Se o Gato não estiver selecionado não irá salvar os dados. Verifique no arquivo **‘Cat_blender.mtl’** gerado se também foi escrito corretamente.

Um detalhe importante já que posicionamos o gato na posição correta, na transformação de **‘rotação’** que estávamos aplicando no OpenGL, é retirar ela ou se não quiser fazer muitas alterações colocar uma rotação de **‘0’** graus.

O que deveria ser feito é configura o nome das matrizes e principalmente da Matriz que esta sendo enviada como **‘model’**.

Resultado



Atividade

Agora carregue mais 5 objetos na cena e se necessário ajuste os arquivos .obj no blender.

Se preferir poderá inserir outras formas na cena, por exemplo um chão, um muro no fundo, etc.

O Trabalho 02 deverá ser entregue até o final da aula de hoje.
Apresentar para o Professor logo que terminar.

Referências e material de apoio

Material do Professor Guilherme Chagas Kurtz, 2023.

GOMES, Jonas; VELHO, Luiz. Computação gráfica. Rio de Janeiro: Impa, 1998.

HEARN, Donald; Baker, M. Pauline. Computer graphics: C version. London: Prentice Hall, 1997.

HETEM JUNIOR, Annibal. Computação gráfica. Rio de Janeiro, RJ: LTC, 2006. 161 p. (Coleção Fundamentos de Informática).

HILL Jr, Francis S. Computer graphics using open GL. New Jersey: Prentice Hall, 2001.

WATT, Alan. 3D computer graphics. Harlow: Addison-Wesley, 2000

Material Prof. Guilherme Chagas Kurtz, 2023.

Thank you for your attention!!



Email: andre.flores@ufn.edu.br